Serving Data with FastAPI

Reindert-Jan Ekker

@rjekker www.codesensei.nl

Overview

Read-only operations

- Serve car data
- Filter cars

Parameters

- Query parameters
- Path parameters

Type hints

- Validation, conversion and more

Return 404 Not Found

Debugging

```python
@app.get("/api/cars")
def get_cars(size, doors):
    ...  # Filter results
```

Passed in the url after ?, separated by &

http://localhost:8000?doors=5&size=m

Query Parameters

Arguments to our function/operation/endpoint
Passed in the url
It’s better to add type hints

```python
@app.get("/api/cars")
def get_cars(size: str, doors: int) -> list:
    ...  # Filter results
```

Type Hints

Ignored by Python
Used by FastAPI for conversion, validation, documentation
(... but not the return type!)
Great editor support

```python
@app.get("/api/cars")
def get_cars(size: str|None = None,
             doors: int|None = None) -> list:
    ...  # Filter results


@app.get("/api/cars")
def get_cars(size: Optional[str] = None,
             doors: Optional[int] = None) -> list:
    ...  # Filter results
```

Optional Query Parameters

By default, query parameters are required
Make them optional by specifying a default value (can be any value, e.g. ”m” or 5)
Don’t forget to make the type hint accept None as well
Before Python 3.10, use Optional

```python
@app.get("/api/cars/{id}")
def car_by_id(id: int) -> dict:

    # Return a car
```

Path Parameters

Name of parameter in URL pattern using curly braces
Matches a part of the URL: http://localhost:8000/api/cars/5
Cannot be made optional

Path and Query Parameters Combined

```
@app.get("/api/cars/{car_id}/trips/")
def trips_for_car(car_id: int,
                  destination: str|None = None) -> dict:
    # Return all trips for this car with given destination
```

Return a 404 Status Code

```
@app.get("/api/cars/{id}")
def car_by_id(id: int) -> dict:
    # Get car...
    if car_does_not_exist:

        raise HTTPException(status_code=404,
                            detail=f"No car with id={id}.")
```

Summary

Read-only operations

Parameters

- Query parameters
- Path parameters

Type hints

- Validation, conversion and more

Return 404 Not Found

Debugging

Up Next: Serving Structured Data Using Pydantic Models